Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине «Проектирование программного обеспечения интеллектуальных систем»
За 1 семестр
Тема: «Пространства имён. Исключения»

Выполнил:
Студент ФЭИС
Группы ИИ-29
Годованный Ю.М.
Проверил:
Монтик Н.С.

Брест 2026

Цель работы: Познакомиться с использованием пространств имён и с механизмом исключений.

Ход работы:

Задание: Создать два пространства имён с классами, имеющими одинаковое название; в каждом классе реализовать метод, который может вызвать исключение; организовать обработку исключений блоками `try` и `catch`; реализовать не менее двух типов исключений (стандартное и пользовательское, определённое в одном из пространств имён); показать, что исключения из разных пространств имён обрабатываются раздельно разными блоками `catch`; при перехвате выводить тип и причину ошибки.

Описание пространств имён и классов исключений:

В предметной области «Студент» созданы два пространства имён: Faculty (студент факультета) и Dormitory (студент-жилец общежития). В каждом определён класс `Student`. Это два разных типа с одинаковым именем: конфликта имён нет, а объекты создаются с указанием полного имени (`Faculty::Student fs`, `Dormitory::Student ds`). Без пространств имён такое определение не компилируется:

```
class Student {};
class Student {};
```

Ошибка компилятора:

```
error: redefinition of 'class Student'
```

1. Пространство имён Faculty.

- `class Student` - поля: `name` - имя, `course` - курс, `grades[SIZE]` - оценки; конструктор и методы `TO_SET_Grade(int i, int value)`, `TO_SHOW()`;
- `TO_SET_Grade` может вызвать два исключения: стандартное `std::out_of_range` при неверном индексе оценки и пользовательское `Faculty::GradeError` при оценке вне диапазона 0..10. Значение изменяется только после всех проверок, поэтому при ошибке объект остаётся корректным;

- `class GradeError : public exception` - пользовательское исключение: поля `message` (причина) и `value` (недопустимое значение), переопределённый метод `what()`, собственные методы `TO_GET_Type()` (тип ошибки) и `TO_GET_Value()`.

2. Пространство имён Dormitory.

- `class Student` - поля: `name` - имя, `room` - номер комнаты; конструктор и методы `TO_MOVE(int newRoom)`, `TO_SHOW()`;
- конструктор выбрасывает `std::invalid_argument` при пустом имени (объект в этом случае не создаётся) и `Dormitory::RoomError` при неверном номере комнаты; `TO_MOVE` выбрасывает `Dormitory::RoomError`, если комнаты с таким номером нет (допустимы номера 1..`MAX_ROOM`);
- `class RoomError : public exception` - пользовательское исключение второго пространства имён: поля `message` и `room`, методы `what()`, `TO_GET_Type()`, `TO_GET_Room()`.

3. Обработка исключений.

- наследование от `std::exception` позволяет перехватить пользовательское исключение и по его собственному типу, и по базовому;
- исключения перехватываются по константной ссылке (`const Faculty::GradeError& e`): объект не копируется;
- общая функция `report(const string& type, const exception& e)` выводит тип ошибки и причину (`e.what()`); так как `what()` виртуальный, вызывается версия, соответствующая реальному типу исключения;
- в блоке из нескольких `catch` порядок важен: сначала перехватываются производные типы (`Faculty::GradeError`, `Dormitory::RoomError`, `std::out_of_range`), и только затем

базовый `std::exception`, иначе он перехватил бы все исключения.

4. Демонстрация работы.

В `main()` показано: создание объектов двух одноимённых классов; стандартные исключения (`out_of_range` при индексе 7, `invalid_argument` при пустом имени); пользовательские исключения (оценка 15 и комната 99) с выводом типа, причины и значения; раздельная обработка четырёх ситуаций в одном блоке `try` с отдельными `catch` для Faculty, Dormitory и стандартных исключений; вывод состояния объектов после ошибок (оценки не изменились, комната изменилась только после корректного вызова `TO_MOVE(20)`).

Код программы:

```
#include <iostream>
#include <string>
#include <exception>
#include <stdexcept>
#include <cstdlib>
#include <clocale>
using namespace std;

namespace Faculty
{

    class GradeError : public exception
    {
    private:
        string message;
        int value;

    public:
        GradeError(int v) : value(v)
        {
            message = "оценка " + to_string(v) + " вне диапазона
0..10";
        }

        const char *what() const noexcept override
        {
            return message.c_str();
        }

        const char *TO_GET_Type() const
        {
            return "Faculty::GradeError";
```

```
        }
        int TO_GET_Value() const { return value; }
    };

    class Student
    {
    public:
        static const int SIZE = 5;

    private:
        string name;
        int course;
        int grades[SIZE];

    public:
        Student(const string &n, int c) : name(n), course(c)
        {
            for (int i = 0; i < SIZE; i++)
            {
                grades[i] = 0;
            }
        }

        void TO_SET_Grade(int i, int value)
        {
            if (i < 0 || i >= SIZE)
            {

                throw out_of_range("индекс " + to_string(i) + "
вне диапазона 0.." + to_string(SIZE - 1));
            }
            if (value < 0 || value > 10)
            {
                throw GradeError(value);
            }
            grades[i] = value;
        }

        void TO_SHOW() const
        {
            cout << "Faculty::Student " << name << ", курс "
                 << course << ", оценки: ";
            for (int i = 0; i < SIZE; i++)
            {
                cout << grades[i] << " ";
            }
            cout << endl;
        }
    };

}

namespace Dormitory
```

```
{
    const int MAX_ROOM = 50;

    class RoomError : public exception
    {
    private:
        string message;
        int room;

    public:
        RoomError(int r) : room(r)
        {
            message = "комнаты " + to_string(r) + " нет
(допустимо 1.." + to_string(MAX_ROOM) + ")";
        }

        const char *what() const noexcept override
        {
            return message.c_str();
        }

        const char *TO_GET_Type() const
        {
            return "Dormitory::RoomError";
        }
        int TO_GET_Room() const { return room; }
    };

    class Student
    {
    public:
        static const int SIZE = 5;

    private:
        string name;
        int room;

    public:
        Student(const string &n, int r) : name(n), room(r)
        {
            if (n.empty())
            {
                throw invalid_argument("имя студента не может "
                                       "быть пустым");
            }
            if (r < 1 || r > MAX_ROOM)
            {
                throw RoomError(r);
            }
        }

        void TO_MOVE(int newRoom)
```

```
        {
            if (newRoom < 1 || newRoom > MAX_ROOM)
            {
                throw RoomError(newRoom);
            }
            room = newRoom;
        }

        void TO_SHOW() const
        {
            cout << "Dormitory::Student " << name
                 << ", комната " << room << endl;
        }
    };

}

void report(const string &type, const exception &e)
{
    cout << "  Тип ошибки: " << type << endl;
    cout << "  Причина: " << e.what() << endl;
}

int main()
{
    system("chcp 65001");
    setlocale(LC_ALL, "ru_RU.UTF-8");

    cout << "=== 1. Пространства имён ===" << endl;

    Faculty::Student fs("Юрий", 2);
    Dormitory::Student ds("Юрий", 12);
    fs.TO_SHOW();
    ds.TO_SHOW();

    cout << endl
         << "=== 2. Стандартные исключения ===" << endl;
    try
    {
        fs.TO_SET_Grade(7, 5);
        cout << "  выполнено без ошибок" << endl;
    }
    catch (const out_of_range &e)
    {
        report("std::out_of_range", e);
    }

    try
    {
        Dormitory::Student bad("", 5);
        cout << "  выполнено без ошибок" << endl;
    }
    catch (const invalid_argument &e)
```

```
{
    report("std::invalid_argument", e);
}

cout << endl
     << "=== 3. Пользовательские исключения ===" << endl;
try
{
    fs.TO_SET_Grade(2, 15);
}
catch (const Faculty::GradeError &e)
{
    report(e.TO_GET_Type(), e);
    cout << "  Значение: " << e.TO_GET_Value() << endl;
}

try
{
    ds.TO_MOVE(99);
}
catch (const Dormitory::RoomError &e)
{
    report(e.TO_GET_Type(), e);
    cout << "  Значение: " << e.TO_GET_Room() << endl;
}

cout << endl
     << "=== 4. Раздельная обработка ===" << endl;
for (int step = 1; step <= 4; step++)
{
    cout << "Шаг " << step << ":" << endl;
    try
    {
        switch (step)
        {
        case 1:
            fs.TO_SET_Grade(9, 1);
            break;
        case 2:
            fs.TO_SET_Grade(1, -3);
            break;
        case 3:
            ds.TO_MOVE(0);
            break;
        case 4:
            ds.TO_MOVE(20);
            break;
        }
        cout << "  ошибок нет" << endl;
    }

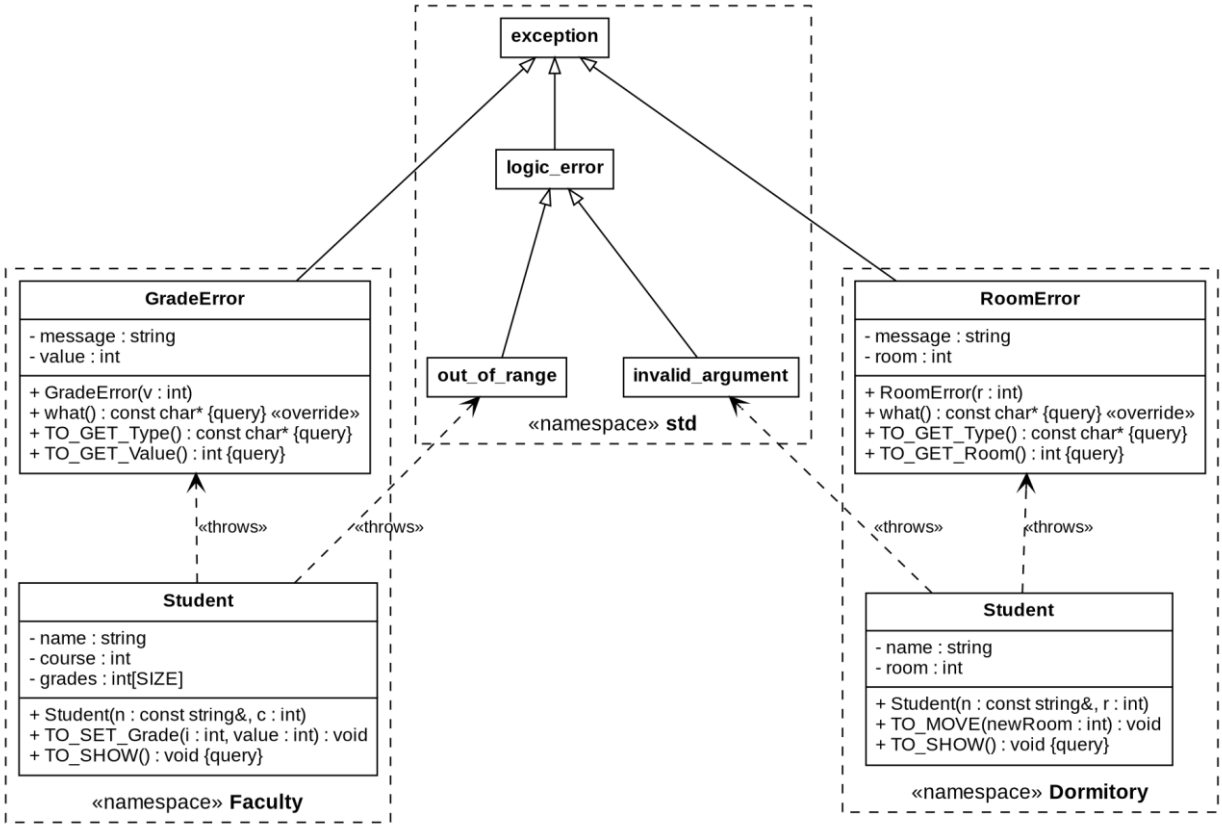
    catch (const Faculty::GradeError &e)
    {
```

```
            cout << "  [catch Faculty]" << endl;
            report(e.TO_GET_Type(), e);
        }
        catch (const Dormitory::RoomError &e)
        {
            cout << "  [catch Dormitory]" << endl;
            report(e.TO_GET_Type(), e);
        }
        catch (const out_of_range &e)
        {
            cout << "  [catch std::out_of_range]" << endl;
            report("std::out_of_range", e);
        }
        catch (const exception &e)
        {
            cout << "  [catch std::exception]" << endl;
            report("std::exception", e);
        }
    }

    cout << endl
         << "Состояние после обработки:" << endl;
    fs.TO_SHOW();
    ds.TO_SHOW();

    return 0;
}
```

UML-диаграмма классов:



Пространства имён показаны пунктирными областями со стереотипом «namespace». Стрелки с пустым треугольным наконечником - наследование от стандартных классов; пунктирные стрелки «throws» показывают, какие исключения выбрасывают методы классов Student.

Результат:

1. Два класса Student в разных пространствах имён:

```
=== 1. Пространства имён ===
Faculty::Student Юрий, курс 2, оценки: 0 0 0 0 0
Dormitory::Student Юрий, комната 12
```

2. Генерация и перехват стандартных исключений:

```
=== 2. Стандартные исключения ===
  Тип ошибки: std::out_of_range
  Причина: индекс 7 вне диапазона 0..4
  Тип ошибки: std::invalid_argument
  Причина: имя студента не может быть пустым
```

3. Пользовательские исключения из разных пространств имён:

```
=== 3. Пользовательские исключения ===
  Тип ошибки: Faculty::GradeError
  Причина: оценка 15 вне диапазона 0..10
  Значение: 15
  Тип ошибки: Dormitory::RoomError
  Причина: комнаты 99 нет (допустимо 1..50)
  Значение: 99
```

4. Раздельная обработка исключений разными блоками catch:

```
=== 4. Раздельная обработка ===
Шаг 1:
  [catch std::out_of_range]
  Тип ошибки: std::out_of_range
  Причина: индекс 9 вне диапазона 0..4
Шаг 2:
  [catch Faculty]
  Тип ошибки: Faculty::GradeError
  Причина: оценка -3 вне диапазона 0..10
Шаг 3:
  [catch Dormitory]
  Тип ошибки: Dormitory::RoomError
  Причина: комнаты 0 нет (допустимо 1..50)
Шаг 4:
  ошибок нет

Состояние после обработки:
Faculty::Student Юрий, курс 2, оценки: 0 0 0 0 0
Dormitory::Student Юрий, комната 20
```

Вывод: Познакомились с пространствами имён и исключениями. Создали два пространства имён (Faculty и Dormitory) с одноимёнными классами Student и показали, что полное имя устраняет конфликт имён. Реализовали стандартные исключения (`std::out_of_range`, `std::invalid_argument`) и два пользовательских класса исключений (`Faculty::GradeError`, `Dormitory::RoomError`), наследуемых от `std::exception` с переопределением `what()`. Показали, что исключения из разных пространств имён обрабатываются раздельно, а блоки `catch` для производных типов должны идти перед блоком для базового типа. Исключения перехватывались по константной ссылке, при каждой ошибке выводились её тип и причина. Построили UML-диаграмму классов.